

Problem Set 11
Comp 301
Spring 2025
Week 12: 12.05.2025 - 16.05.2025

Instructions:

- Submit your answers to the Learnhub PS11 assignment until May 17th Saturday, at 23.59.
- Please use the code boilerplate, which includes several tests for you to see if your code is correct.
- Save your code and pdf file, zip it as ID_username.zip with your ID and username (Example: 1234567_johndoe21.zip), and submit this ZIP file.
- Read the questions carefully. Good luck!

Problem 1: ¹ In IREF, why are variables in environment bound to integers rather than expressed values, as in EREF?

Problem 2: In IREF we can build recursive procedures by assignment. The following pieces of code can be replaced by one another. Explain why this is the case.

1)

```
letrec times4(x) = if zero?(x)
                    then 0
                    else -((times4 -(x,1)), -4)
in (times4 3)
```

2)

```
let times4 = 0
in begin
  set times4 = proc (x)
    if zero?(x)
    then 0
    else -((times4 -(x,1)), -4);
  (times4 3)
end
```

Hint: Think of why the following PROC code piece generates an error and fails to operate recursively.

```
let f = proc(x) if zero?(x) then 0 else -((f -(x, 1)), -1)
in (f 3)
```

¹EOPL p.121 Exercise 4.15

Problem 3: ² Modify the implementation of `proc-exp` to allow multiple arguments. Also change the implementation of `call-exp` to accept multiple operands.

Hint: You have implemented multi-argument procs before. Now try to do it in IREF.

New grammar of `proc-exp` and `call-exp`

$$\begin{aligned} \textit{Expression} &:= \textit{proc}(\{\textit{Identifier}\}^{*(\cdot)}) \textit{Expression} \\ &:= (\textit{Expression} \{\textit{Expression}\}^*) \end{aligned}$$

Example

```
let x = 5 in let y = 6 in
  let z = proc (a,b) -(a,b) in (z x y)
=> returns (num-val -1)
```

Recommended Changes:

- **Note:** Carefully read the instructions commented in the code as well. More detailed information is given in the comments.
- Change `proc` datatype and implement `extend-env*`, which extends environment with multiple var val pairs at once, in `data-structures.rkt`
- Change `apply-env` to handle the `extend-env*` in `environments.rkt`
- Change the implementations of grammars of `proc-exp` and `call-exp` accordingly in `lang.rkt`
- Change the implementation of `proc-exp`, `call-exp` inside `value-of` and also change `apply-procedure` to evaluate multi-argument procedures in `interp.rkt`.

²EOPL Exercise 4.17

Example

```
let x = 5, y = 6 in -(x,y)
=> returns (num-val -1)
```

Recommended Changes:

- **Note:** Carefully read the instructions commented in the code as well. More detailed information is given in the comments.
- Change the implementation of grammar of let-exp accordingly in `lang.rkt`
- Change the implementation of let-exp in `interp.rkt`
- **Hint:** You can use previously implemented `extend-env*`

Problem Set 11 Solutions

Comp 301

Spring 2025

Problem 1: ¹ In IREF, why are variables in environment bound to integers rather than expressed values, as in EREF?

In IREF every time a new variable is created it is created in the store so we save an integer to point to that location, but in EREF it could be any expressed value and references.

Problem 2: In IREF variables are mutable. Therefore we can introduce the variable simply as a name to be stored in the saved environment of a procedure and then change it to the function that we want to define recursively. In this way, the name that is in the saved environment will point to the function itself.

¹EOPL p.121 Exercise 4.15